Архитектура для высоких нагрузок: кэширование, масштабирование, асинхронность

Платформа товарных знаний на базе мультимодального GraphRAG

Домашнее задание 9. Курс OTUS “ИИ-архитектор”

Содержание

Контекст и оговорка **1**

1. Почему требование в заданном виде невыполнимо **2**
1.1. 200 мс не вмещают генерацию 2
1.2. 10 000 RPS генерации не покупаются 2
1.3. Проверка на здравый смысл 3

2. Встречное требование **3**

3. Кэширование **4**
3.1. Четыре уровня, и только один из них семантический 4
3.2. Алгоритм семантического кэша 5
3.3. Три ограничения, которых нет в типовой схеме 5
3.4. Эмбеддер для кэша - отдельная статья расходов 6

4. Горизонтальное масштабирование **6**
4.1. Узкое место меняется: теперь это префилл, а не полоса памяти 6
4.2. Балансировка не может быть round-robin 7
4.3. Автомасштабирование: KEDA там, где оно работает 7

5. Асинхронная обработка **8**

6. Geo-DNS и edge: разбор конфликта **9**

7. Что из этого делает требование выполнимым **10**

8. Чего эта работа не доказывает **10**

Контекст и оговорка

Репозиторий проекта: https://github.com/SandalAndrey/otus_ai_architect

Заказчик - интернет-магазин коллекционных масштабных моделей: более 15 000 позиций, около 400 марок, порядка 600 производителей. Действующие требования:

2 RPS и p95 не более 5 секунд (NFR-1), одна арендованная A100 40 ГБ (ADR-0019), закрытый контур без обращений к внешним API.

Задание предлагает сценарий роста: 10 000 RPS и задержка менее 200 мс. Это **в 5000 раз больше нагрузка при в 25 раз более жёстком бюджете времени**. Сценарий синтетический: Заказчик таких требований не ставил, и в проектную документацию эти цифры не переносятся.

Работа устроена так: сначала разбор, почему требование в заданном виде невыполнимо ни за какие деньги, затем встречное требование - такое же агрессивное, но физически достижимое, - и уже под него архитектура.

1. Почему требование в заданном виде невыполнимо

1.1. 200 мс не вмещают генерацию

Бюджет времени на ответ разобран в ADR-0010 и состоит из лимитов вывода по ролям агентов.

Роль	Лимит вывода
Супервизор	32 токена
Векторный агент	48 токенов
Агент обхода графа	64 токена
Составитель	200 токенов
Верификатор	64 токена

Класс A - 344 токена последовательно, класс B - 360. При 108 токенах в секунду на поток (ADR-0019) это **3.2 и 3.3 секунды только на генерацию**, без извлечения, переранжирования и проверок.

Для сравнения: один входной рубеж guardrails по ADR-0010 имеет бюджет 300 мс, то есть в полтора раза больше всего отпущенного задания.

Сократить ответ до 200 мс нельзя: при 108 ток/с это 21 токен. Двадцать один токен - это не ответ на вопрос о различиях двух выпусков модели, это обрывок фразы.

Вывод: 200 мс - бюджет отдачи готового ответа, а не бюджет его порождения. Любая архитектура, укладывающаяся в 200 мс, устроена так, что подавляющее большинство запросов до модели не доходит. Это не оптимизация поверх архитектуры, это и есть архитектура.

1.2. 10 000 RPS генерации не покупаются

Пропускная способность одной A100 считается из тех же лимитов: 2160 токенов в секунду на карту (108 на поток при 20 потоках), 360 токенов на запрос класса B.

```
6 запросов в секунду на карту
```

Доля попаданий в кэш	RPS до модели	Карт A100	Аренда, руб/мес
0 процентов	10 000	1667	300 060 000

Доля попаданий в кэш	RPS до модели	Карт A100	Аренда, руб/мес
50 процентов	5 000	834	150 120 000
90 процентов	1 000	167	30 060 000
99 процентов	100	17	3 060 000

Цена аренды - 180 000 руб в месяц за карту с хостом (docs/07, проверено по Yandex Cloud и Cloud.ru в октябре 2026). Таблица даёт главное число всей работы:

Один процент попаданий в кэш стоит 3 миллиона рублей в месяц.

Это не риторика, а линейная зависимость: 1 процент от 10 000 RPS - это 100 запросов в секунду, то есть 16.7 карт. Отсюда следует, что на такой нагрузке кэш перестаёт быть способом ускорения и становится способом существования.

1.3. Проверка на здравый смысл

10 000 RPS - это 864 миллиона обращений в сутки. На 15 000 позиций каталога получается около 57 000 вопросов в сутки на каждую позицию. Такого спроса на коллекционные масштабные модели не существует ни в одной стране.

Отмечаю это не чтобы отказаться от задания, а чтобы зафиксировать: требование синтетическое не только технически, но и коммерчески. Поэтому встречное требование ниже тоже синтетическое - но хотя бы внутренне непротиворечивое.

2. Встречное требование

Исходное требование смешивает под одним SLO два разных класса трафика. Разделим их - это и есть основное проектное решение работы.

Класс	Доля	RPS	SLO	Путь
1. Повторный или типовой вопрос	93 процента	9 300	p95 не более 200 мс	Кэш и предрасчёт, GPU не задействован
2. Новый вопрос	6.5 процента	650	p95 не более 5 с, TTFT 1.5 с	Синхронная генерация со стримингом
3. Тяжёлый запрос	0.5 процента	50	До 60 с, уведомление о готовности	Очередь, асинхронно

Точка входа по-прежнему держит 10 000 RPS, и 200 мс остаются - но как SLO класса 1, а не всего трафика. Классы 2 и 3 сохраняют бюджеты действующих требований проекта, потому что физика не меняется от роста нагрузки.

Откуда доля 93 процента. Складывается из двух источников, и ни один из них не является обычным кэшем ответов:

- **Предрасчёт головы распределения, около 70 процентов.** 15 000 позиций, умноженные на 15-20 типовых шаблонов вопросов (масштаб, прототип, тираж, отличия выпусков, аналоги), дают порядка 250 тысяч ответов. Это конечное число, и его можно сгенерировать офлайн - один раз на версию индекса.
- **Семантический кэш хвоста, около 23 процентов.** Переформулировки тех же вопросов, опечатки, разный порядок слов.

Класс 3 - это запросы, которые и сегодня не укладываются в 5 секунд: сравнение десятка позиций, разбор загруженного пользователем PDF-каталога, подбор по размытому описанию с обходом графа на глубину больше двух.

3. Кэширование

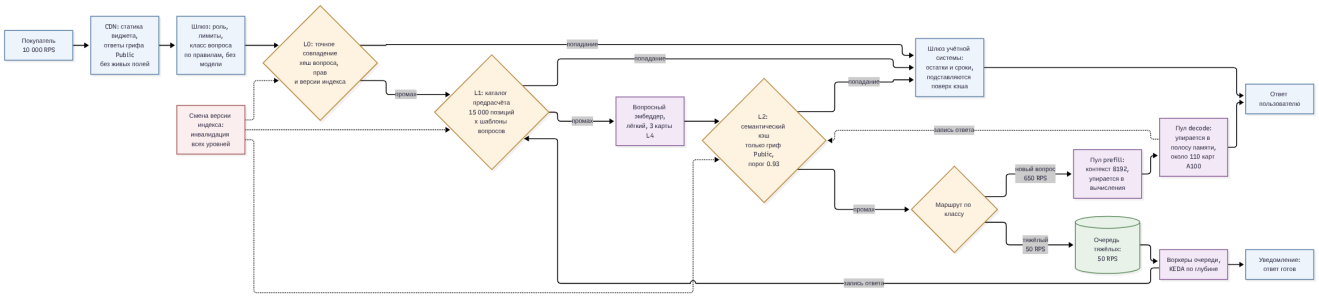


Рис. 1: Контур на 10 000 RPS: CDN и шлюз, три уровня кэша по возрастанию стоимости, маршрут по классу запроса, разделённые пулы префилла и декода, очередь тяжёлых запросов, инвалидация всех уровней по версии индекса. Исходник - `highload-architecture.mmd`, картинка в полном разрешении

3.1. Четыре уровня, и только один из них семантический

Уровень	Что хранит	Ключ	Время ответа
CDN	Статика виджета, ответы графа Public без живых полей	URL	10-30 мс
L0, точный	Ответы на дословно совпавшие вопросы	Хеш текста, прав и версии индекса	2-5 мс
L1, предрасчёт	Каталог сгенерированных офлайн ответов	Позиция и шаблон вопроса	2-5 мс
L2, семантический	Ответы на близкие по смыслу вопросы	Вектор вопроса	20-40 мс

Порядок обращения именно такой: от дешёвого к дорогому. L2 стоит на порядок дороже остальных, потому что требует эмбеддинга входящего вопроса.

L1 - главный уровень, и его обычно забывают. Семантический кэш наполняется тем, что уже спросили; предрасчёт наполняется тем, что спросят. На холодном старте и после каждой переиндексации семантический кэш пуст, а предрасчёт - нет. Без L1 первые часы после выпуска вся нагрузка идёт в модель, и 1667 карт из раздела 1.2 перестают быть гипотезой.

3.2. Алгоритм семантического кэша

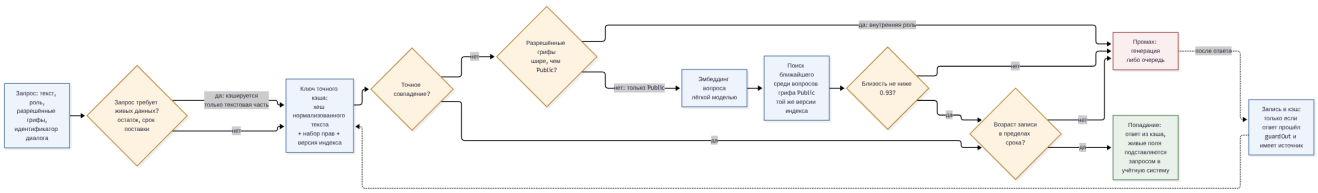


Рис. 2: Алгоритм семантического кэша: отсев запросов с живыми данными, ключ точного кэша из текста, прав и версии индекса, отказ от семантического поиска для внутренних ролей, порог близости, проверка срока, запись только прошедших guardOut ответов. Исходник - `semantic-cache.mmd`, картинка в полном разрешении

1. **Нормализация.** Приведение регистра, свёртка пробелов, нормализация артикулов и наименований - **тем же кодом, что и при индексации**. Разные реализации нормализации на двух путях - уже опознанный в проекте дефект (ADR-0012), и кэш добавляет для него третье место.
2. **Построение ключа L0:** хеш от нормализованного текста, набора разрешённых грифов и мажорной версии индекса. Совпадение - сразу к шагу 6.
3. **Проверка грифа.** Если роль видит больше, чем Public, семантический поиск **не выполняется** - промах. Обоснование в разделе 3.3.
4. **Эмбеддинг вопроса** лёгкой моделью (раздел 3.4) и поиск ближайшего среди вопросов той же версии индекса.
5. **Порог близости 0.93 по косинусу.** Ниже порога - промах. Порог назначен, а не выведен: подбирается на золотом датасете так, чтобы доля ложных попаданий была нулевой, а не малой.
6. **Проверка срока.** Запись старше срока жизни - промах.
7. **Подстановка живых полей.** Остатки и сроки поставки берутся из учётной системы в момент ответа, из кэша они не выдаются никогда (раздел 3.3).
8. **Запись в кэш** выполняется только для ответов, прошедших `guardOut` и содержащих ссылку на источник. Ответ без источника нарушает NFR-2.4, и закэшированный такой ответ нарушает его многократно.

3.3. Три ограничения, которых нет в типовой схеме

Это место, где наш проект отличается от примера из документации Azure.

Близость по смыслу не даёт права доступа. Вопросы “какая у нас розничная цена на эту модель” и “какая у нас закупочная цена на эту модель” отличаются одним словом и будут близки в любом эмбеддинге. Первый доступен покупателю, второй - только закупщику и администратору (BR-01). Семантическое попадание через границу грифа - это утечка коммерческой тайны, а утечка грифа у нас блокирующий критерий приёмки с нулевым порогом.

Отсюда правило: **семантический кэш работает только для грифа Public**. Для внутренних ролей разрешён лишь точный кэш L0, где права входят в ключ. Это снижает общий процент попаданий, но внутренние роли дают малую долю трафика, а цена ошибки несопоставима.

Живые данные не кэшируются никогда. NFR-4.1 требует свежести остатков в 15 минут, и достигается это тем, что остатки вообще не индексируются - шлюз запрашивает их в момент ответа. Кэш хранит текстовую часть ответа с метками подстановки, живые поля дописываются поверх. Закэшированный остаток - это обещание наличия товара, которого нет.

Версия индекса входит в ключ. Переиндексация обесценивает кэш целиком, и это не ошибка, а требование: ответ, построенный по прошлой версии каталога, после смены псевдонима коллекции становится неверным. Следствие неприятное - после каждого выпуска индекса кэш холодный, и нагрузка на модель скачкообразно растёт. Поэтому L1 пересчитывается офлайн **до** переключения псевдонима, а не после.

3.4. Эмбеддер для кэша - отдельная статья расходов

Поиск по L2 требует эмбеддинга каждого входящего вопроса. На 10 000 RPS это само по себе вычислительная задача, которую в оценках обычно теряют.

Модель	Вычислений на запрос	Запросов/с на карту L4	Ка
bge-m3, 568M (наш индексный эмбеддер)	36 ГФЛОП	около 825	12
Лёгкая вопросная модель, 100M	6 ГФЛОП	около 4700	3

Брать для кэша индексный эмбеддер не нужно: L2 сравнивает вопросы с вопросами, а не вопросы с документами, и общего векторного пространства с индексом ему не требуется. Лёгкая модель экономит девять карт.

Оговорка: цифры получены из счёта операций (2 x параметры x токены) при 30 ТФЛОП/с полезных на L4 и длине вопроса 32 токена. Это оценка порядка, а не замер.

4. Горизонтальное масштабирование

4.1. Узкое место меняется: теперь это префилл, а не полоса памяти

В ADR-0019 всё упиралось в полосу памяти: генерация читает все веса на каждый токен. На 20 потоках это верно. На 3250 одновременных запросах - нет.

Префилл (разбор промпта) - задача **вычислительная**, декод (порождение токенов) - задача **по полосе памяти**. На нашем пути пять вызовов модели, и промпты у них длинные: составитель и верификатор получают извлечённые фрагменты.

Вызов	Токенов промпта
Супервизор	300
Векторный агент	400
Агент обхода графа	600
Составитель	2500

Вызов	Токенов промпта
Верификатор	2500
Итого	6300

Вычислений: `2 x 14e9 x 6300 = 176 ТФЛОП` на запрос. A100 даёт около 172 ТФЛОП/с полезных (312 пиковых при КПД 0.55; веса в INT4 разжимаются в FP16 для вычислений). Значит **одна карта делает примерно один префилл в секунду**.

Что считаем	Карт на 650 RPS
Декод, по полосе памяти	109
Декод, по видеопамяти под KV-cache	90
Префилл, без переиспользования префикса	669
Префилл, при переиспользовании 75 процентов	168

Префилл дороже декода вшестеро. Это переворачивает вывод ADR-0019 и объясняет, почему на высокой нагрузке пулы префилла и декода разделяют: смешав их, мы получаем блокировку коротких декодов длинными префиллами, то есть рост p95 без роста загрузки.

Переиспользование префикса - главный рычаг. Фрагменты каталога повторяются между запросами о той же позиции, а составитель и верификатор видят одни и те же фрагменты. Кэш префиксов в движке инференса превращает 669 карт в 168. Это тот самый KV-кэш из лекции, и интеграция с движком здесь не удобство, а четырёхкратная разница в счёте.

Итого на класс 2: **около 280 карт, порядка 50 млн руб в месяц**. Цифра большая, но она физически реальна - 280 A100 существуют и арендуются, в отличие от 200 мс на генерацию.

4.2. Балансировка не может быть round-robin

Три причины, и все три специфичны для инференса.

Инстансы не stateless. В карте живёт KV-кэш диалога, а по NFR-1.6 диалог хранит до 60 обменов. Запрос, ушедший на другую карту, заставляет пересчитать префикс заново - это тот же префилл, который мы только что считали самым дорогим. Значит нужна привязка по идентификатору диалога.

Привязка не должна быть жёсткой. Если карта перегружена, лучше заплатить пересчётом префикса, чем встать в очередь. Правило: привязка по хешу диалога, отказ от привязки при длине очереди выше порога.

Запросы несопоставимы по стоимости. Вопрос класса A и разбор PDF отличаются на порядок. Балансировка по числу соединений обманывает: одно соединение может стоить как пятьдесят. Распределять нужно по оценке стоимости - классу вопроса и длине промпта, - а она известна на шлюзе, до модели.

4.3. Автомасштабирование: KEDA там, где оно работает

KEDA масштабирует поды по внешней метрике. Для очереди это естественно:

```yaml
# Скелет, не применялся. Очередь тяжёлых запросов, класс 3.
triggers:
  - type: prometheus
    metadata:
      query: sum(inference_queue_depth{class="heavy"})
      threshold: "20"          # запросов на один под
minReplicaCount: 2             # не ноль, см. ниже
maxReplicaCount: 40
cooldownPeriod: 600            # 10 минут: карта дорого стартует
```

Для синхронного пути всё иначе, и это главное ограничение автомасштабирования на GPU.

Холодный старт измеряется минутами. Загрузка 10.1 ГБ весов, захват CUDA-графов, прогрев - от одной до трёх минут на уже существующем узле. Если узел надо получить у облака, добавьте ещё 5-10 минут. Реактивное масштабирование по факту роста очереди опаздывает на всю эту величину: к моменту готовности нового пода всплеск закончился, а SLO нарушено.

Отсюда три правила:

1. **Никакого scale-to-zero на GPU.** `minReplicaCount` держит тёплый пул, рассчитанный на ночной минимум. Карта, простаивающая ночью, дешевле нарушенного SLO утром.
2. **Масштабирование по опережающему признаку.** Не по длине очереди, а по времени ожидания в ней и по производной трафика: порог срабатывает раньше, чем очередь станет видимой пользователю.
3. **Расписание важнее реакции.** Нагрузка интернет-магазина предсказуема посуточно и по дням недели. Плановое масштабирование закрывает 90 процентов случаев, KEDA остаётся на остальные.

Прогрев обязателен. Под, принявший трафик сразу после старта, отвечает в разы медленнее: не прогреты CUDA-графы, пуст кэш префиксов. Проба готовности должна отдавать “готов” только после прогоночных запросов, иначе балансировщик направит реальный трафик в холодную карту.

5. Асинхронная обработка

Класс 3 - запросы, которые не укладываются в 5 секунд и сегодня. На текущей нагрузке их просто нет: такие вопросы отклоняются либо деградируют. На 10 000 RPS их 50 в секунду, и для них нужен отдельный путь.

Элемент	Решение
Классификация	На шлюзе, правилами: число сравниваемых позиций, наличие вложения, глубина обхода. Модель для этого не вызывается - иначе асинхронный путь начинается с синхронного
Очередь	Отдельная очередь на класс, приоритет ниже синхронного пути

Элемент	Решение
Идемпотентность	Ключ запроса - хеш содержимого; повтор не порождает второй генерации
Обратное давление	При длине очереди выше порога новые тяжёлые запросы получают отказ с оценкой времени, а не встают в неё. Очередь, растущая неограниченно, - способ превратить деградацию в отказ
Доставка результата	Уведомление в интерфейсе, ответ лежит в L1 и доступен по ссылке
Срок жизни	24 часа либо до смены версии индекса, что раньше

Тяжёлые запросы выгоднее всех остальных. Их результат почти всегда переиспользуем: сравнение двух конкретных выпусков одинаково для всех, кто его спросит. Поэтому ответы класса 3 пишутся не в L2, а сразу в L1, то есть пополняют предрасчёт.

6. Geo-DNS и edge: разбор конфликта

Пункт помечен в задании как опциональный, и у нас он сталкивается с двумя требованиями: NFR-3.5 обязывает хранить данные на территории РФ, а закрытый контур исключает обработку за его пределами.

Это не повод пропустить пункт. Граница проходит не по географии, а по грифу.

Что	Можно ли вынести на edge	Почему
Статика виджета	Да	Данных не содержит
Ответы грифа Public без живых полей	Да	Публичны по определению, уже отданы на сайте
Остатки и сроки поставки	Нет	Гриф Internal, плюс требование свежести 15 минут
Ответы грифов Internal и Confidential	Нет	NFR-3.1 и BR-01
Эмбеддинг вопроса	Нет	Вопрос покупателя может содержать ПДн; маскирование выполняется внутри контура
Сам индекс и модель	Нет	Закрытый контур, ADR-0009

Получается edge-слой из одного уровня - CDN с ответами грифа Public. Это немного, но именно он закрывает заметную часть из 93 процентов класса 1 и снимает сетевую

задержку, которой в 200 мс иначе не находится места.

Последний запрещающий пункт стоит назвать прямо: **вынести на edge сам инференс нельзя**, и дело не только в праве. Карта с моделью в каждом регионе - это полный комплект из 280 карт на регион, умноженный на число регионов.

7. Что из этого делает требование выполнимым

Мера	Что даёт	Чем подтверждается
Предрасчёт головы распределения (L1)	70 процентов трафика мимо GPU	250 тысяч ответов - конечное число, считается офлайн
Семантический кэш хвоста (L2)	Ещё 23 процента	Порог 0.93, только гриф Public
Разделение SLO по классам	200 мс там, где достижимо	Генерация не укладывается в 200 мс физически
Разделение пулов префилла и декода	Отсутствие блокировки декода префиллом	Префилл дороже декода вшестеро
Переиспользование префикса	168 карт вместо 669	Фрагменты каталога повторяются между запросами
Очередь для класса 3	Тяжёлые запросы не занимают синхронный пул	50 RPS, приоритет ниже
Тёплый пул вместо scale-to-zero	Нет провалов SLO на старте подов	Холодный старт 1-3 минуты
CDN для грифа Public	Снятие сетевой задержки	Единственное, что разрешено выносить

Итоговая стоимость класса 2: около 280 карт A100, порядка 50 млн руб в месяц. Это и есть честный ответ на вопрос “сколько стоит 10 000 RPS”: не “дорого”, а конкретная сумма, которую можно сопоставить с выручкой.

8. Чего эта работа не доказывает

Доля попаданий 93 процента - предположение. Оно опирается на оценку распределения вопросов, которой у нас нет: реального трафика не существует, а гипотеза о 30 000 запросов в месяц сама не подтверждена Заказчиком. Публичные оценки семантического кэширования в похожих продуктах дают 20-50 процентов, и наши 93 держатся целиком на предрасчёте L1. Если распределение окажется плоским - а для коллекционного каталога с длинным хвостом редких марок это вероятно, - предрасчёт не покроет и половины, и стоимость вырастет кратно.

Порог близости 0.93 назначен, а не выведен. Подбирается на золотом датасете, но датасет для этого не предназначен: он проверяет качество ответов, а не разделимость вопросов.

Расчёт префилла - оценка по счёту операций. КПД 0.55 и 75 процентов переиспользования префикса взяты как правдоподобные величины. Замеров нет ни тех, ни других; разброс здесь способен изменить число карт втрое.

Ничего из описанного не спроектировано для реализации. Это разбор гипотетического сценария. Действующие требования проекта остаются прежними: 2 RPS, p95 5 секунд, одна карта. Переносить цифры этой работы в проектную документацию нельзя - они относятся к другой системе.

Главное, что работа показала. Два вывода пережили бы и настоящий рост нагрузки. Первый: на такой нагрузке кэш - это не оптимизация, а несущая конструкция, и один процент попаданий стоит 3 млн руб в месяц. Второй: узкое место смещается с полосы памяти на вычисления, то есть вывод ADR-0019, верный при 20 потоках, перестаёт быть верным при 3250.